

Search, Fetch, or Answer: Commercial Search APIs Reshape the Retrieval Budget of LLM Agents Without Changing Their Accuracy

Anonymous ACL submission

Abstract

Commercial search APIs are the de facto retrieval layer for tool-using LLM agents, yet they are typically evaluated as static ranked-list components. We argue that a search API is better understood as a *decision surface*: its snippets, rankings, and URL pool jointly shape whether an agent searches again, opens a page, answers from snippets, or abstains. We test this view on the 100-query SEALQA-HARD subset by running the same GPT-5.4 agent with a two-tool interface (`search_web`, `fetch_page`) over three production providers: Brave, Tavily, and Firecrawl. An LLM judge labels every visible URL across 6,909 surface judgments (2,202 / 2,481 / 2,226 per provider) as gold-supporting, contradicting, supporting-the-model-answer, or garbage, enabling a per-query oracle of which URLs the agent *should* have opened. The three providers produce nearly identical exact-match accuracy (21, 21, 23 / 100) but achieve it through fundamentally different retrieval pipelines: Brave’s surface lifts the URL-pool ceiling to 33% via rich `extra_snippet` blocks; Tavily concentrates 48% of gold-supporting URLs at rank one, raising the fetch-recall of supporting URLs to 31%; Firecrawl exposes the sparsest surface but compensates with broad, mostly-blind fetching that wins 20% of no-gold-visible queries. Fetch policy ROI varies from +26 percentage points (Tavily) to −13pp (Firecrawl) on the same model. We use the contradict-to-gold URL ratio (Brave 0.92, Tavily 1.87, Firecrawl 2.59) as an answer-independent surface-quality measurement and argue that retrieval-budget evaluation, not top-*k* relevance, is the right axis for assessing search providers for agentic systems.

1 Introduction

Search-augmented LLM agents now ship in consumer products, deep-research assistants, and enterprise workflows. They all share the same ar-

chitecture: a model calls a search API, reads results, optionally opens pages, and answers. The search API is treated as a fungible commodity: swap Brave for Tavily and you supposedly get the same agent. This paper argues that view is wrong, but *not in the way prior work has measured*. Final answer accuracy varies surprisingly little across providers (Section 4). What varies dramatically is how the agent allocates its retrieval budget: which URLs it opens, which it ignores, when it searches again, and how often it answers correctly from snippets alone. These behavioral differences are larger and more practically important than the accuracy delta.

We frame search APIs as *decision surfaces*. The provider’s snippets, extra-snippet blocks, URL pool, and ranking order are the only signals an agent sees before committing fetch tokens. Two providers with similar top-*k* URL relevance can induce completely different fetch policies: one rewards reading the top result, another rewards reading lower-ranked results, a third rewards skipping fetches entirely. Final accuracy can mask this because the underlying model adapts – but the cost, latency, and failure-mode distribution do not.

Our contribution is fourfold:

- Agentic retrieval evaluation with a per-URL oracle.** We instrument an agent with separate `search_web` and `fetch_page` tools, then run an LLM-judge oracle over every visible URL, whether the model opened it or not (§3). This yields a per-query confusion matrix of SMART, MISSED, BLIND, and NO-OP decisions for each of three commercial providers.
- Empirical mechanism analysis across Brave, Tavily, and Firecrawl** on the 100-query SEALQA-HARD subset, totaling 6,909 surface judgments (6,519 snippet-only

083
084

085
086
087
088
089
090
091
092

093
094
095
096
097

098
099
100
101
102
103
104
105

106

107
108
109
110
111
112
113
114
115
116

117
118
119
120
121
122
123
124
125
126
127
128

rows plus 350 page-visible rows) and 300 query-provider trajectories (§4).

3. **Three findings that survive scrutiny:** providers achieve indistinguishable accuracy through different pipelines (§4.2); fetch-policy return-on-investment is provider-dependent (§4.3); and per-URL contradict-to-gold ratio is a clean, answer-independent surface-quality metric that varies $2.8\times$ across providers (§4.4).
4. **Actionable design guidance:** a fixed agent fetch policy is suboptimal across providers, and the right policy can be predicted from surface-level signals before any model run (§6).

We deliberately avoid framing this as a provider leaderboard. The leaderboard view (“provider *A* beats provider *B* on EM”) would conclude there is little to choose between Brave, Tavily, and Firecrawl on hard QA. The decision-surface view, by contrast, gives a structural reason *why* accuracy converges and identifies the policy and cost levers that providers actually move.

2 Related Work

Search-augmented and agentic QA. Retrieval-augmented generation (Lewis et al., 2020; Izacard and Grave, 2021) and search-augmented agents (Shuster et al., 2022; Schick et al., 2023; Yao et al., 2023) have largely measured end-to-end answer quality. BrowseComp (OpenAI, 2025) stresses agent capability on hard browsing tasks. Our work shares the hard-QA setting but inverts the focus: the controlled variable is the search provider, not the agent.

Retrieval evaluation and judge oracles. Top-*k* relevance metrics (Thakur et al., 2021) dominate retriever evaluation but capture only the static URL ranking, not the agent’s decisions. LLM-as-judge methodology (Zheng et al., 2023) has been used to grade answers; we extend it to grade every visible URL, producing an oracle of which results an ideal agent would have opened. To our knowledge, the smart/foolish/missed-skip matrix this enables has not previously been instantiated across commercial providers with a fully judged URL pool.

Hard QA benchmarks. SEALQA-HARD (vtllms, 2025) contains 254 web-search questions designed to be unsolvable from parametric memory and to expose conflicting or unhelpful retrieved evidence. We use its publicly released subset, sampled to 100 queries via deterministic stratified sampling on freshness, search_results, and topic (see Appendix A).

Commercial search APIs as research artifacts. Brave Search (Brave Software, 2025), Tavily (Tavily, 2025), and Firecrawl (Firecrawl, 2025) are widely used in agent stacks but have not, to our knowledge, been compared as decision surfaces under a controlled agentic protocol. Existing comparisons report top-*k* URL relevance or downstream EM only.

3 Methodology

Figure 1 summarizes the experiment architecture.

3.1 Agent Architecture

The agent exposes two tools.

search_web(query) returns up to ten ranked results per provider. Each result carries title, URL, domain, snippet, and any provider-native extra_snippet blocks. Crucially, no page content is included; the agent sees only what the provider’s API natively returns.

fetch_page(document_id) fetches and extracts the full text of a previously-surfaced URL via Jina Reader (Jina AI, 2024). The model receives extracted markdown, fetch status, extractor name, and character count. The fetch backend is identical across providers, so any differences in fetched content are attributable to the URL pool, not the extractor.

At each turn the model may search again, fetch any previously-seen URL, answer with FINAL ANSWER: $\langle x \rangle$, or abstain. The maximum iteration budget is 10, the maximum results per search is 10, and the model is GPT-5.4 (temperature 0, max output tokens 4096). The system prompt, user-query wrapper, and tool schema are identical across providers; only the search backend changes.

3.2 Providers and Dataset

We use the v2 endpoints of three production providers: Brave Web Search, Tavily Search, and Firecrawl Search (configuration in Appendix B).

129
130
131
132
133
134
135
136

137
138
139
140
141
142
143
144

145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175

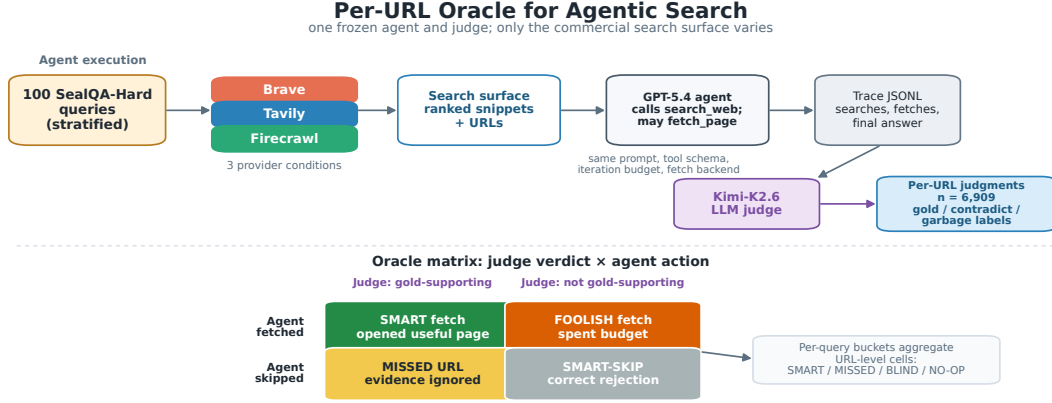


Figure 1: Experiment architecture and the per-URL oracle. The same 100 SEALQA-HARD queries are sent through three provider conditions (Brave, Tavily, Firecrawl), holding the GPT-5.4 agent, prompt, tool schema, iteration budget, and fetch backend constant. Each agent run produces a JSONL trace recording every `search_web` call, every `fetch_page` call, and the final answer. The Kimi-K2.6 LLM judge then emits a per-URL judgment at the snippet-only surface (for every visible URL) and at the page-visible surface (for every URL the agent actually fetched), totaling 6,909 judgments across the three provider conditions. Joining the judge verdict with the agent’s per-URL action yields the 2×2 oracle confusion matrix shown at the bottom; the per-query buckets in Table 3 aggregate these per-URL cells.

Every provider is configured with `country=US`, English language settings, and no provider-side page scraping; we want to evaluate the search surface, not the provider’s optional fetch layer. The exact HTTP request payloads we sent are reproduced in Appendix C.

The dataset is 100 queries from SEALQA-HARD (vtllms, 2025), selected by deterministic proportional stratified sampling over freshness $\times$ search_results (SealQA’s “conflicting / unhelpful” label) $\times$ topic (seed 20260509). The sample is 57% conflicting and 43% unhelpful; covers all six SealQA topics (Science & Tech 27, Sports 22, Entertainment 22, Other 12, Politics 9, History & Geography 8); 43% slow-changing, 32% never-changing, 25% fast-changing; and 61% from before-2024, 16% 2024, 23% 2025. The worst stratum drift versus the 254-row source distribution is +3.3pp. Full per-cell allocations and the multi-label question-type breakdown are in Appendix A.

3.3 Per-URL Oracle Judging

For every URL the agent saw, whether or not it opened it, we run an LLM judge (Kimi-K2.6, temperature 0). The judge inspects exactly the same XML fragment the agent saw at decision time (title, URL, domain, snippet, extra_snippets if any), plus the gold answer, the question, and the model’s final answer. The schema is

```
{
  "contains_gold_answer": bool,
  "supports_model_answer": bool,
  "contradicts_gold_answer": bool,
  "is_garbage": bool,
  "answer_span": str,
  "confidence": float
}
```

URLs the agent fetched are judged twice: once on the *snippet-only* surface (matching what the agent saw before deciding to fetch) and once on the *page-visible* surface (matching what the agent saw after fetching). This yields two ceilings: the snippet-surface ceiling (what was answerable before fetching) and the URL-pool ceiling (what was answerable after fetching from any URL in the pool). Total judgments: 6,909 across providers (2,202 / 2,481 / 2,226), with 6,869 producing a valid structured judgment; the 40 invalid rows are timeouts or schema-parse failures excluded from analysis (Section 7).

A measurement caveat must be stated up front: Brave’s snippet-only surface includes *multiple* extra_snippet blocks per result (median ≈ 3 blocks of 200–400 chars each), while Tavily and Firecrawl expose a single snippet only. This is a provider-design choice, not an artifact of our pipeline, and the agent and the judge see exactly the same text. But it means any judgment on the snippet surface for Brave is made on $\approx 5 \times$ more text per URL than for Tavily or Firecrawl. We

Metric	Brave	Tavily	Firecrawl
Exact Match	21	21	23
Token F1 (avg)	0.270	0.261	0.282
Answered	98	97	96
Abstained	2	3	4
Avg search calls	2.29	2.74	2.51
Avg fetch calls	1.02	1.30	1.28
% queries fetched	65	76	81
Total tokens (M)	5.96	5.42	5.80
Avg tokens/query	59,627	54,156	57,979

Table 1: Headline metrics over 100 SEALQA-HARD queries with the agentic two-tool architecture. EM differences are within Wilson 95% CIs and not significant.

report the per-URL contradict-to-gold ratio (Section 4.4) as the answer-independent surface metric not subject to this confound (both fields use the same per-URL surface).

3.4 The Smart / Missed / Blind Matrix

We partition the 100 queries per provider into four mutually exclusive, exhaustive buckets:

- SMART: ≥ 1 gold-supporting URL was visible *and* the model fetched at least one gold-supporting URL.
- MISSED: ≥ 1 gold-supporting URL was visible *and* the model fetched none of them (possibly fetched something else, possibly fetched nothing).
- BLIND: no gold-supporting URL was visible *and* the model fetched ≥ 1 URL.
- NO-OP: no gold-supporting URL was visible *and* the model fetched none.

Throughout, “gold-supporting URL visible” means the LLM judge labeled at least one URL in the surface (snippet or fetched page) as `contains_gold_answer`. SMART and MISSED share the same denominator (queries with gold visible); their EM gap is the cleanest empirical estimate of the value of fetching when gold is on the surface.

4 Results

4.1 Headline accuracy is indistinguishable

Table 1 shows the surface-level summary. Exact-match accuracy is statistically indistinguishable across providers (Wilson 95% CIs overlap heavily at $n=100$). Tavily issues 20% more searches

per query than Brave; Firecrawl fetches pages on 25% more queries than Brave. If a paper stopped here, it would conclude that provider choice does not matter much.

4.2 Finding 1: Three pipelines, one accuracy

Table 2 shows three orthogonal pipelines.

Brave: rich snippet surface, broad URL pool, low fetch recall. Brave’s URL-pool ceiling is 33 queries – nearly double the others. Of those 33, the agent answers 12 correctly (36%), and an additional 9 correct answers come from queries the judge marked unsupported, for 21 EM in total. Brave returns 87 unique supporting URLs across the 33 ceiling-supported queries (≈ 2.6 supporting URLs per query on average) but the agent fetches only 9 of them (10.3% recall). Brave’s gold-supporting URLs are roughly uniformly distributed across all 10 ranks (12 at rank 1, 17 at rank 3, 9 at rank 9; see Figure 3).

Tavily: top-rank concentration, high fetch recall, high conditional capture. Tavily places **48% of its gold-supporting URLs at rank 1** (15/31), versus 12% for Brave and 11% for Firecrawl. The agent’s natural “open the top result” tendency therefore converts into 30.8% fetch recall, three times Brave’s. Among the 24 ceiling-supported queries, Tavily’s agent captures 12 (50%), the highest conditional capture rate of any provider. The remaining 9 correct answers come from beyond-ceiling queries, giving 21 EM total.

Firecrawl: sparse surface, broad fetching, beyond-ceiling wins. Firecrawl’s URL-pool ceiling is only 19 queries; the agent captures 7 of them (37%). Yet Firecrawl reaches 23 EM total – the highest of the three – because 16 of those 23 correct answers come from queries the judge marked unsupported. This “beyond-ceiling EM” (20% on the 81 no-gold-visible queries) is double the rate of Brave and Tavily (each 12–13%). Firecrawl’s agent fetches on 81% of queries and 54% of those fetches go to rank 1, but most rank-1 URLs do not contain gold by snippet. Wins on the beyond-ceiling queries come from broadly fetching URLs whose snippets were sparse but whose page content held the answer.

The three providers thus converge on ~ 21 – 23 EM through (i) reading rich snippets, (ii) reading rank 1, and (iii) reading broadly: three retrieval styles with very different cost and failure-

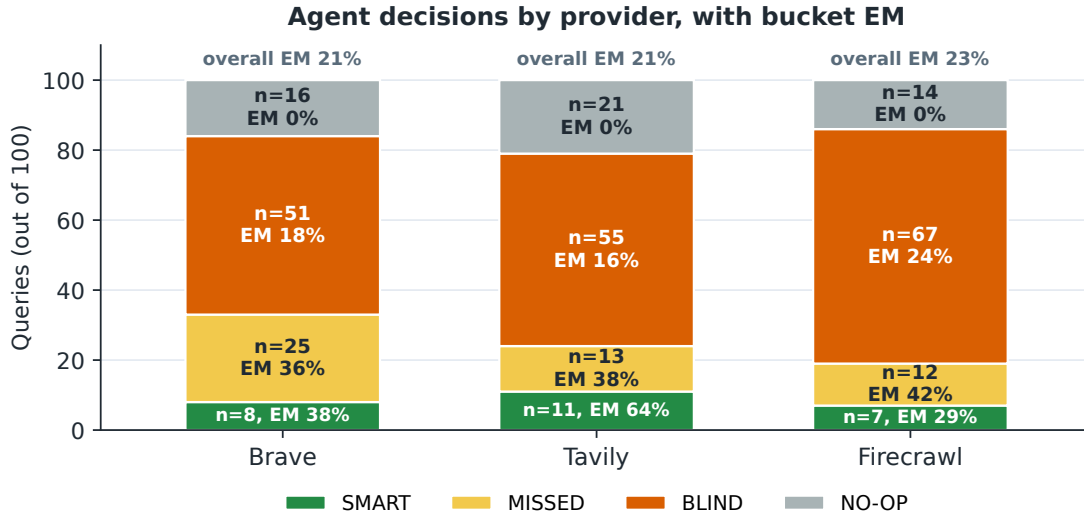


Figure 2: Smart / Missed / Blind / No-op partition of the 100 queries per provider, with EM% inside each bucket. The three providers reach ≈ 21 – 23 EM through structurally different bucket distributions: Brave has the largest evidence-visible share (SMART + MISSED = 33), Tavily concentrates correct answers in a small high-payoff SMART bucket (64% EM, the highest cell in the dataset), and Firecrawl spends most queries in BLIND with the dataset’s only above-average blind-fetch EM (24% vs. Brave 18%, Tavily 16%). NO-OP queries (no gold-supporting URL ever judged visible, no fetches) are unsolvable for every provider on this dataset.

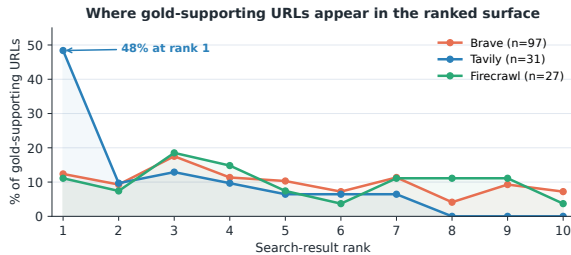


Figure 3: Rank distribution of gold-supporting URLs per provider, normalized within each provider. Tavily concentrates 48% of its gold-supporting URLs at rank 1, versus 12% (Brave) and 11% (Firecrawl). This is the structural reason Tavily’s “open the top result” agent behavior converts at $3\times$ Brave’s fetch recall.

mode profiles. Figure 2 visualizes the resulting smart/missed/blind/no-op partition.

4.3 Finding 2: Fetch-policy ROI is provider-dependent

Table 3 shows the SMART/MISSED/BLIND/NO-OP matrix. The SMART–MISSED gap measures the lift from opening a gold page when one is on the surface:

- **Tavily:** +26pp. Opening Tavily’s top result, when it is gold-supporting, is highly valuable.
- **Brave:** +2pp. Brave’s rich snippet surface already conveys the answer; the page adds lit-

tle.

- **Firecrawl:** –13pp. On Firecrawl, fetching the gold page actually correlates with *lower* EM than skipping it. The sample is small ($n=7$ SMART) but the direction is consistent with Firecrawl’s beyond-ceiling EM in BLIND: the model does best on Firecrawl by fetching broadly into the URL pool, not by trusting the snippet-judged signal.

The same generic agent policy of “search, optionally fetch a result, answer” has positive, near-zero, and negative ROI across the three providers. This is a structural fact about how each provider’s surface aligns with the model’s natural decision heuristics.

4.4 Finding 3: A clean, answer-independent surface-quality metric

The headline contains a circularity hazard: a URL labeled `supports_model_answer` $\wedge$ $\neg$ gold can only exist when the model’s answer differed from gold, making any “misleading content” count partly tautological with EM. We therefore use the answer-independent `contradicts_gold_answer` field as the surface-quality metric of record. Per URL u , with `contradicts(u)` and `containsGold(u)` the judge’s

Surface-quality and oracle metric	Brave	Tavily	Firecrawl
Snippet-only URLs judged	2,095	2,339	2,085
URLs containing gold answer (%)	4.6%	1.3%	1.3%
URLs contradicting gold (%)	4.2%	2.5%	3.4%
Garbage URLs	22	34	9
Queries with ≥ 1 gold URL visible (URL-pool ceiling)	33	24	19
Queries with ≥ 1 contradicting URL visible	33	27	35
Queries with only-contradicting evidence	20	18	27
Queries with no judged evidence either way	47	58	54
% of gold-supporting URLs placed at rank 1	12%	48%	11%
Unique supporting URLs in pool	87	39	33
Useful fetches (model $\cap$ oracle)	9	12	7
Oracle fetch precision	8.9%	9.6%	5.6%
Oracle fetch recall	10.3%	30.8%	21.2%
EM gold visible (conditional capture)	36%	50%	37%
EM no gold visible (beyond-ceiling EM)	13%	12%	20%
EM from beyond-ceiling queries (absolute count)	9	9	16

Table 2: Surface-quality landscape and oracle outcomes per provider. “URL-pool ceiling” is the count of queries where the LLM judge labeled at least one URL in the surface (snippet *or* fetched page) as containing the gold answer. The last block shows that the three providers reach ≈ 21 –23 EM via different routes: Brave and Tavily mostly capture queries on the ceiling, Firecrawl’s wins come overwhelmingly from queries the judge marked unsupported.

Bucket (n / EM%)	Brave	Tavily	Firecrawl
SMART	8 / 38%	11 / 64%	7 / 29%
MISSED	25 / 36%	13 / 38%	12 / 42%
BLIND	51 / 18%	55 / 16%	67 / 24%
NO-OP	16 / 0%	21 / 0%	14 / 0%
Total (n sums to 100)	100	100	100
SMART – MISSED (Δ pp)	+2	+ 26	– 13

Table 3: EM by smart/misssed/blind/no-op bucket. NO-OP here is the strict partition complement: no gold visible *and* the model fetched nothing. Such queries are essentially unsolvable on this dataset (0/51 EM across all providers). The SMART–MISSED gap is the value of opening a gold-supporting page when one is visible.

two binary labels, we define the contradict-to-gold ratio

$$r_{c:g} = \frac{|\{u : \text{contradicts}(u)\}|}{|\{u : \text{containsGold}(u)\}|}$$

aggregated across all snippet-judged URLs visible to a provider’s agent. $r_{c:g}$ measures how often the surface delivers contradicting content relative to supporting content, independent of whether the model used either.

The $2.8\times$ spread (Table 4) is the largest answer-independent provider effect in our data. It is also predictive: in the cross-tab of gold-visible $\times$ contradicting-visible (Table 5), adding contradicting URLs to a gold-visible surface drops EM by 9pp on Brave (40% $\rightarrow$ 31%) and 45pp on

Provider	Contradict:gold ratio
Brave	0.92 (89 / 97)
Tavily	1.87 (58 / 31)
Firecrawl	2.59 (70 / 27)

Table 4: Per-URL contradict-to-gold ratio (snippet-only surface). Brave’s surface is roughly balanced; Firecrawl returns 2.6 contradicting URLs for every supporting one. See Figure 4 for a visualization.

Surface state (n / EM%)	Brave	Tavily	Firecrawl
Gold, no contradictions	20 / 40%	15 / 67%	11 / 36%
Gold + contradictions	13 / 31%	9 / 22%	8 / 38%
Only contradictions	20 / 5%	18 / 0%	27 / 15%
Neither (no judged evidence)	47 / 17%	58 / 16%	54 / 22%

Table 5: EM under all four answer-independent surface states. Tavily collapses (67% $\rightarrow$ 0%) when contradictions appear; Firecrawl is uniquely resilient on no-evidence queries (22%).

Tavily (67% $\rightarrow$ 22%), but shifts +2pp on Firecrawl (36% $\rightarrow$ 38%, $n=8$, not statistically meaningful). Tavily’s surface is the most lethal under contamination, since its model relied on the top result and polluting nearby ranks proves costly.

5 Case Studies

Two queries exemplify the pipelines (a third is in Appendix I).

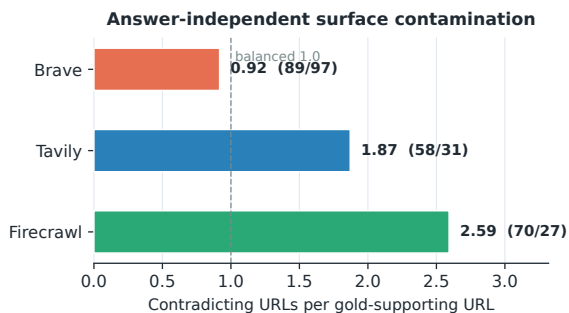


Figure 4: Per-URL contradict-to-gold ratio per provider (Table 4). This ratio is answer-independent: it does not depend on what the agent answered, only on how the surface aligns with gold. The $2.8\times$ spread is the largest answer-independent provider effect in our data.

Case A: Tavily’s smart fetch wins (SEALHARD_6F8F541710FE). According to 2021 world population data, how many continents account for less than 10% of the world’s population? Gold = 5. Brave surfaced Statista, Wikipedia World Population, and Worldometer; the agent fetched Worldometer’s population-by-region page and answered 4 (wrong). Tavily surfaced StatisticsTimes (rank 1), Facebook (rank 2), and Wikipedia “List of continents and continental subregions by population” (rank 3). The agent fetched the Wikipedia list page and answered 5 (correct). Firecrawl placed the same Wikipedia page at rank 1 but the agent did not fetch, answering 4 from snippets (wrong). The canonical page existed in all three result sets; Tavily’s rank ordering induced the right fetch.

Case B: Brave answers without fetching (SEALHARD_5668B93DD088). Who was the second non-Asian men’s soccer player to pass 100 international goals...? Gold = Lionel Messi. Brave’s two searches surfaced multiple Wikipedia pages with extra_snippet blocks naming Messi as a recent 100-goal passer. Zero fetches; answered *Lionel Messi* (correct). Tavily’s single search returned a Tehran Times article whose snippet read “Ronaldo becomes second player to reach 100 international goals”; with no extra_snippet disambiguation, the agent answered *Cristiano Ronaldo* (wrong). Firecrawl ran four searches and one fetch to reach the correct answer at higher cost. Brave’s snippet richness substituted for fetching.

6 Discussion

6.1 The right fetch policy is provider-dependent

A widely-used heuristic in agent stacks is “fetch the top-3 results.” Our results show this is provider-dependent in three ways. On Tavily, fetching rank 1 is high-ROI (+26pp on gold-visible queries) but fetching ranks 2–3 may add contradicting context (the 67%→22% collapse in Table 5). On Brave, fetching is largely wasted token cost; the snippet surface already conveys what the model needs in $\geq 36\%$ of gold-visible cases without any fetch. On Firecrawl, fetching is structurally cheap insurance: the snippet surface underreports the URL pool’s actual support, and broad fetching recovers queries the agent would otherwise lose.

This suggests an *adaptive* policy: condition fetch budget on provider identity (cheap heuristic) or on prompt-visible surface signals (more general). The contradict-to-gold ratio at the per-provider level is a strong correlate of optimal fetch breadth.

6.2 Why does accuracy converge?

Two compensating mechanisms explain why EM is similar across very different surfaces. **Snippet richness substitutes for fetching:** Brave’s extra_snippet blocks let the model answer from the surface alone on a large fraction of gold-visible queries (Case B). **Broad fetching substitutes for snippet richness:** Firecrawl’s sparse snippets force broad fetching (81% of queries), recovering queries the snippet judge cannot evaluate. The same agent “compensates its way” to similar EM in multiple regimes; if you remove richness, the model fetches more, and if you remove fetch budget, the rich-surface providers cope but the sparse-surface ones collapse.

6.3 Implications for evaluation

Provider evaluation should report at minimum: (i) per-URL contradict-to-gold ratio; (ii) rank distribution of supporting URLs; (iii) the smart/missed/blind matrix on a fixed agent; (iv) cost per supported answer. Top- k relevance, the dominant retrieval metric today, is insufficient because it ignores how an agent will actually use the surface.

7 Limitations

The URL-pool ceiling is a lower bound on true URL-pool support. Of the 6,869 valid judgments, only 350 page_visible rows (101 / 125 / 124 for Brave / Tavily / Firecrawl) received a page-text judgment; the remaining 6,519 snippet-only rows were judged on title + URL + snippet (+ extra_snippets for Brave) alone. A URL whose snippet was sparse but whose underlying page contained gold can only be discovered as gold-supporting if the model happened to fetch it. A full page-level oracle over all 5,565 unique visible URLs (1,700 / 2,035 / 1,830 per provider) is left to future work; Firecrawl’s beyond-ceiling EM (Section 4.2) is direct evidence that the current ceiling understates true URL-pool support.

Single judge model. All 6,869 valid judgments come from Kimi-K2.6 at temperature 0. We did not run a second-judge calibration. The contradict-to-gold ratio and the SMART/MISSED/BLIND matrix are robust to judge bias in the sense that the same judge is applied uniformly to all providers, but absolute ceiling rates may be conservative or liberal.

Single agent model. All trajectories are GPT-5.4. Provider-specific findings (e.g., Tavily’s rank-1 ROI) may shift with a different model. We selected GPT-5.4 because it is a current-generation production model widely used in tool-use deployments.

Observational, not causal. Surface state and query difficulty co-vary; we cannot claim that the surface *caused* the trajectory differences without a replay experiment in which the same agent state is fed a different provider’s results. We leave such counterfactual replays to future work.

Sample size. 100 queries per provider yields Wilson 95% CIs of ± 8 pp for proportions near 0.5. Several cell-level differences in Table 3 do not reach significance at $n=100$. The qualitative pattern (Tavily+26, Brave+2, Firecrawl−13) is robust; the precise magnitudes are not.

40 invalid judgments (6 Brave, 17 Tavily, 17 Firecrawl), comprising timeouts or JSON schema-parse failures, were excluded from analysis. These are roughly uniformly distributed across queries and unlikely to bias provider comparisons.

Cost is the binding constraint on scale. The full study consumed roughly 17.0M agent tokens across 300 trajectories, plus 6,909 LLM-judge calls, plus three commercial search-API plans and a Jina Reader subscription. End-to-end this was the limiting factor for scaling to more queries, more providers, or more agent models; 100 queries is the largest pool we could fully judge under our budget. Releasing the trace and judgment dataset (Section 8) reduces this marginal cost for follow-up work.

Three commercial providers only. Google Custom Search, Bing, Exa, and Serper are not evaluated. The proposed methodology (snippet+page oracle, contradict:gold ratio) generalizes; we expect similar across-pipeline divergence with other providers.

8 Released Resources

To support replication we release two artifacts (URLs anonymized for review, public at camera-ready):

- **Dataset** (HuggingFace): 300 agent trajectories, 6,909 LLM-judge rows with the v2 schema, deterministic per-query metrics, and the smart/missed/blind partition. Schema, license, and a usage notebook are included.
- **Code** (GitHub): the agent loop, two-tool implementation, trace schema, judge prompt, provider adapters, the Jina Reader page-fetch layer, and the scripts that reproduce every number in this paper.

Given API keys, `run_phase1.py` reproduces the 100-query runs end-to-end; all deterministic offline metrics regenerate from the released JSONL files without any model calls.

9 Conclusion

Commercial search APIs are usually treated as interchangeable input components for retrieval-augmented agents. We instead treat them as decision surfaces that reshape how the agent allocates its retrieval budget. On 100 hard SEALQA-HARD queries, three production providers converge on near-identical EM (21–23 / 100) through three structurally different pipelines (rich snippets, top-rank concentration, broad fetching), fetch-policy ROI varies from +26 to −13 pp, and per-URL

contradict-to-gold ratios vary $2.8\times$. Retrieval-budget evaluation, not top- k relevance or end-to-end accuracy alone, is the right axis for choosing and routing search providers for agentic systems.

References

- Brave Software. 2025. Brave Search API reference. <https://api-dashboard.search.brave.com/api-reference/>.
- Firecrawl. 2025. Firecrawl search API documentation. <https://docs.firecrawl.dev/api-reference/endpoint/search>.
- Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880.
- Jina AI. 2024. Jina Reader: URL to LLM-friendly input. <https://r.jina.ai/>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- OpenAI. 2025. BrowseComp: A simple yet challenging benchmark for browsing agents. <https://openai.com/research/browsecomp>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*.
- Kurt Shuster, Jing Xu, Mojtaba Komeili, Da Ju, Eric Michael Smith, Stephen Roller, Megan Ung, Moya Chen, Kushal Arora, Joshua Lane, and 1 others. 2022. BlenderBot 3: A deployed conversational agent that continually learns to responsibly engage. In *arXiv preprint arXiv:2208.03188*.
- Tavily. 2025. Tavily Search API documentation. <https://docs.tavily.com/>.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.
- vllms. 2025. SealQA: Raising the bar for reasoning in search-augmented language models. <https://huggingface.co/datasets/vllms/sealqa>.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.

A Dataset Stratification

We sample 100 queries from the 254-row SEALQA-HARD subset by deterministic proportional stratified sampling on freshness $\times$ search_results $\times$ topic with seed 20260509. Table 6 reports the full per-stratum distribution and the percentage-point drift versus the source.

Stratum	Sample	Δ vs. src
<i>Freshness</i>		
fast-changing	25 (25%)	−0.2 pp
slow-changing	43 (43%)	−0.7 pp
never-changing	32 (32%)	+0.9 pp
<i>Search-result quality</i>		
conflicting	57 (57%)	+0.3 pp
unhelpful	43 (43%)	−0.3 pp
<i>Topic</i>		
Science & Tech	27 (27%)	+0.2 pp
Sports	22 (22%)	0.0 pp
Entertainment	22 (22%)	+0.4 pp
Others	12 (12%)	−0.2 pp
Politics	9 (9%)	−0.1 pp
History & Geog.	8 (8%)	−0.3 pp
<i>Effective year</i>		
before 2024	61 (61%)	+2.3 pp
2024	16 (16%)	−3.3 pp
2025	23 (23%)	+1.0 pp
<i>Question type (multi-label)</i>		
advanced reasoning	72 (72%)	−0.4 pp
entity/event disamb.	61 (61%)	+2.7 pp
temporal tracking	13 (13%)	−0.8 pp
cross-lingual	5 (5%)	−0.5 pp
false-premise	2 (2%)	−2.3 pp

Table 6: Distribution of the 100-query SEALQA-HARD sample. “ Δ vs. src” is the percentage-point difference between the 100-query sample fraction and the 254-row source distribution; worst drift is +3.3pp. Question types are multi-label, so column sums exceed 100.

B Provider API Configurations

Summary: all providers configured with country=US, English language, no provider-side page scraping. Brave: count=10, safesearch=moderate, result_filter=web.

Tavily: search_depth=basic, topic=general, include_raw_content=false. Firecrawl: limit=10, sources=[web], scrapeOptions omitted. Identical Jina Reader fetch backend (r.jina.ai) for all three.

C Programmatic Request Payloads

For full reproducibility we reproduce the exact HTTP request the agent sends per provider on each search_web call.

Brave Web Search.

```
GET https://api.search.brave.com/res/v1/web/search
  ?q=<model-generated query>
  &count=10
  &offset=0
  &country=US
  &search_lang=en
  &ui_lang=en-US
  &safesearch=moderate
  &spellcheck=true
  &text_decorations=false
  &result_filter=web
Headers:
  X-Subscription-Token: <BRAVE_KEY>
  Accept: application/json
  Accept-Encoding: gzip
```

Tavily Search.

```
POST https://api.tavily.com/search
Headers:
  Authorization: Bearer <TAVILY_KEY>
  Content-Type: application/json
Body:
{
  "query": "<model-generated query>",
  "search_depth": "basic",
  "topic": "general",
  "max_results": 10,
  "include_answer": false,
  "include_images": false,
  "include_image_descriptions": false,
  "include_favicon": false,
  "include_raw_content": false
}
```

Firecrawl Search (v2).

```
POST https://api.firecrawl.dev/v2/search
Headers:
  Authorization: Bearer <FIRECRAWL_KEY>
  Content-Type: application/json
Body:
{
  "query": "<model-generated query>",
  "limit": 10,
  "sources": ["web"],
  "country": "US",
  "timeout": 60000,
  "ignoreInvalidURLs": false
}
```

Page fetch (Jina Reader, all providers).

```
GET https://r.jina.ai/<encoded URL>
Headers:
  Authorization: Bearer <JINA_KEY>
  Accept: text/markdown
```

The agent never receives raw HTML; the page surface is always markdown extracted by Jina Reader. This isolates the variable of interest (provider URL pool + rank + snippet) from any page-extraction variance.

D Auto-Fetch Architecture Comparison

For context we ran the same 100 queries under an *auto-fetch top-k* architecture where the agent’s search_web call returns snippet + extracted page text for every visible URL (no separate fetch_page tool). Table 7 compares the two architectures.

Architecture / Provider	EM	Avg Tok	Max Tok
<i>Auto-fetch top-3 (baseline)</i>			
Brave	21	49,229	602,701
Tavily	22	84,753	1,768,647
Firecrawl	22	44,431	397,227
<i>Agentic-fetch (main)</i>			
Brave	21	59,627	303,462
Tavily	21	54,156	305,474
Firecrawl	23	57,979	380,646

Table 7: Auto-fetch top-3 vs agentic-fetch architectures. Agentic fetch reduces Tavily token tails by 5.8× at the max with no EM loss.

Agentic-fetch caps the maximum token tail (Tavily: 1.77M → 305K, −83%) with essentially no change in EM. The mean token usage moves in opposite directions for the providers, reflecting how each surface induces a different fetch budget. Detailed numbers ship in results/provider_comparison/.

E Smart / Missed / Blind Confidence Intervals

Bucket (k/n)	Brave	Tavily	Firecrawl
SMART	3/8: 38% [14–69]	7/11: 64% [35–85]	2/7: 29% [8–64]
MISSED	9/25: 36% [20–55]	5/13: 38% [18–64]	5/12: 42% [19–64]
BLIND	9/51: 18% [10–30]	9/55: 16% [9–28]	16/67: 24% [15–30]
NO-OP	0/16: 0% [0–19]	0/21: 0% [0–15]	0/14: 0% [0–21]

Table 8: Wilson 95% CIs for the bucket EM rates in Table 3. Tavily’s SMART–MISSED gap is the only cell-level effect that approaches separation; others should be read as exploratory.

F Agent System Prompt

The full agent system prompt is reproduced verbatim from the released code.

You are an assistant that answers factual questions using web search.

You have access to:

- search_web: query string. Returns results with document_id, title, URL, snippet.
- fetch_page: a document_id from a prior search_web result. Returns extracted page text + fetch metadata.

<constraints>

- Every final answer must be grounded in retrieved evidence. Never answer from parametric knowledge alone.
- If sources conflict, prefer the most recent, most authoritative (official > news > blogs).
- For time-sensitive questions, note the date of the evidence.

</constraints>

<workflow>

1. Understand the query.
2. Formulate concise keyword queries.
3. Evaluate snippets.
4. fetch_page when snippets are insufficient/ambiguous/stale.
5. Iterate as needed.
6. Produce the final answer.

</workflow>

<output_format>

- Prefix with "FINAL ANSWER: ".
- Return only the factual answer.
- If unanswerable, respond exactly: "FINAL ANSWER: I cannot determine the answer from available sources."

</output_format>

A neutral worked example follows in the released code and is omitted here for space.

G LLM Judge Prompt

The judge prompt is also reproduced verbatim:

You are an evidence-support judge for a search-augmented QA evaluation.

Judge whether ONE retrieved document contains enough evidence to answer the user question with the known gold answer, whether it supports the model's final answer, whether it contradicts the gold answer, and whether the content is mostly unusable garbage.

Rules:

- Judge ONLY the provided document (title, URL, domain, snippet, extra snippets, extracted page text).
- Do not use outside knowledge.
- The gold answer is for evaluation.
- contains_gold_answer = the document fully contains evidence to answer with the gold answer (NOT mere string match).
- gold_answer_in_snippets = snippet fields alone suffice.
- gold_answer_in_extracted_page =

extracted_page text alone suffices.

- supports_model_answer = document supports the model's answer when different from gold.
- contradicts_gold_answer = the document asserts facts that conflict with the gold answer.
- is_garbage = empty/binary/extraction debris/blocked/boilerplate; if extracted_page is empty/failed and snippets carry no evidence, mark garbage.
- answer_span / gold_snippet_span / gold_extracted_page_span = short verbatim spans.
- Return JSON only.

H Representative Judge Samples

We show one real judge sample per outcome class. URLs are truncated for space; full samples are in the released dataset.

Gold-supporting (smart-fetch evidence). *Q: Which is the oldest university in continuous operation in Italy that began as a faculty-led institution? Gold = University of Padua. URL: aleteia.org/2018/01/10/the-5-oldest-universities-in-continuous-operation-and-their (rank 10). The judge returned contains_gold_answer=true with answer_span: "The University of Padua was founded as a school of law in 1222 in Padua, Italy, and remains a prestigious institution today with over 40,000 students. Famous ancient alumni: ... Nicolaus Copernicus, ..."*

Contradicts gold (misleading-content evidence). *Q: What is the supercomputer with the fastest speed outside the US? Gold = HPC6; model answered JUPITER. URL: nvidianews.nvidia.com/news/nvidia-powers-europes-fastest-supercomputer (rank 3). contradicts_gold_answer=true with answer_span: "JUPITER is on track to be Europe's first exascale supercomputer." (Confidence 1.0.) The page is well-formed; it simply asserts a different answer than gold.*

Garbage page (failed extraction). *Q: Which country has won the most bronze medals in the World Pastry Cup? Gold = Italy; model answered Italy. URL: pastryteamusa.com/the-sweet-pursuit-of-perfection-a-history-of-the-coupe-du-monde (rank 8). is_garbage=true with garbage_reason: "Extracted page is a 404 error and remaining snippets"*

do not provide useful evidence for the gold or model answer.”

shots of the live viewer are bundled in the camera-ready supplementary material.

Supports model’s wrong answer (the EM-circularity bucket from §4.4). *Q:* According to ITU, which country had the highest number of mobile cellular subscribers per 100 people worldwide? *Gold* = Sweden; *model* answered Finland. URL: www.itu.int/ITU-D/ict/publications/wtdr_99/material/wtdr99s.pdf (rank 9). supports_model_answer=true, contains_gold_answer=false with answer_span: “Finland is in some ways a curious place to be the world leader in mobilephone penetration.” (Confidence 0.8.) This is exactly the answer-dependent label whose circularity we avoid by using contradicts_gold_answer instead in Section 4.4.

I Case C: Same page, different counts

Q: How many times in history has the PDC World Darts Championship final featured two English players? *Gold* = 14. (SEALHARD_370DE35828D1.) All three providers placed the Wikipedia “PDC World Darts Championship” page at rank 1, and the agent fetched exactly that URL in every provider’s run. Yet the final answers were 15 (Brave), 15 (Tavily), and 14 (Firecrawl, correct). The fetched evidence was identical; the surrounding rank-2/3 results and snippets differed (Brave: dartsnews.com; Tavily: dartskins.co.uk; Firecrawl: darts501.com). This case demonstrates that the surface continues to influence reasoning *after* fetching: the rest of the prompt context changes how the model counts entries in the same Wikipedia table.

J Trace Viewer

The released code ships an interactive trace dashboard (scripts/trace_dashboard.py, served at <http://127.0.0.1:8765>) that loads any of the JSONL trace files and any judge JSONL, then renders per-query timelines (every search call, every retrieved result with snippet, every fetch call with the extracted page text, the model’s tool calls, and the final answer) together with the per-URL judge verdicts. A static one-shot renderer (scripts/render_trace.py) emits the same view as a standalone HTML page for archival, which is useful for case studies (Section 5) and for reviewing failure modes. Screen-